

Multi-Query / Self-Query Retriever

1. Multi-Query Retriever

1. Multi-Query Retriever란?

Multi-Query Retriever는 하나의 질문을 여러 개의 다양한 질문(Query)으로 변환한 후 각각 검색을 수행하여 검색 결과를 합치는 기법이다.

즉,

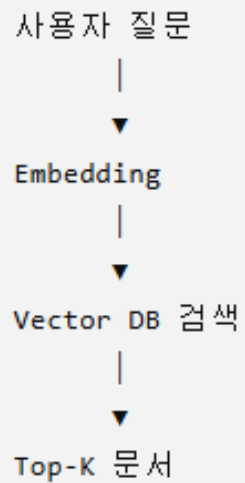
질문 하나 → 여러 개의 질문 생성 → 각각 검색 → 결과 통합

이라는 과정을 수행한다.

LLM이 질문을 다양한 표현으로 바꾸어 검색하기 때문에, 하나의 질문만으로는 찾기 어려운 문서까지 검색할 수 있다.

2. 기존 검색 방식

기존 RAG에서는 사용자의 질문을 그대로 검색한다.



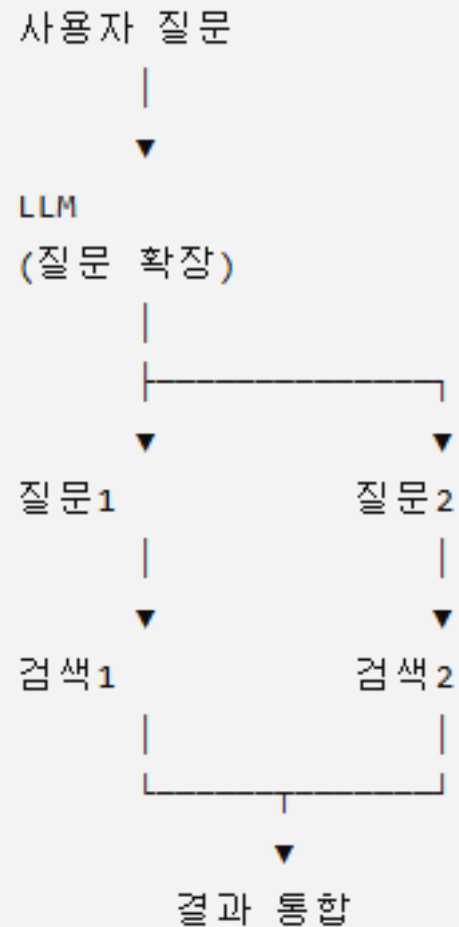
예)

사용자 질문

LangChain Agent란?

검색도 동일한 문장으로 수행된다.

3. Multi-Query 방식



예)

사용자 질문

LangChain Agent란?

LLM이 생성

LangChain Agent의 개념은?

Agent의 역할은?

Tool Calling이란?

LangChain Agent 사용법은?

이 네 개의 질문으로 각각 검색한다.

4. 왜 성능이 좋아질까?

사람마다 같은 내용을 다양한 방식으로 질문한다.

예)

RAG란?

또는

Retrieval-Augmented Generation

또는

LLM 검색 증강

Vector DB에는

검색 증강 생성 (RAG)은...

으로 저장되어 있을 수도 있다.

하나의 질문만 검색하면 놓칠 수 있지만

여러 질문을 검색하면

더 많은 관련 문서를 찾을 수 있다.

5. 기존 검색 vs Multi-Query

기존 검색	Multi-Query
질문 1개	질문 여러 개
검색 1회	검색 여러 번
검색 범위 좁음	검색 범위 넓음
속도 빠름	조금 느림
정확도 보통	정확도 높음

6. 동작 과정

사용자 질문



LLM



질문 4~5개 생성



각 질문 임베딩



Vector Search



결과 통합



중복 제거



최종 문서 반환

7. 예제

사용자 질문

벡터 데이터베이스란?

LLM 생성

벡터 DB란?

Vector Database의 역할은?

임베딩 데이터 저장소란?

FAISS와 Chroma는 무엇인가?

Vector Search란?

이 질문들로 각각 검색한다.

8. 간단한 Python 예제

Python

```
from langchain_openai import ChatOpenAI
from langchain_chroma import Chroma
from langchain_openai import OpenAIEmbeddings
```

```
llm = ChatOpenAI(model="gpt-4.1-mini")
```

```
embedding = OpenAIEmbeddings(
    model="text-embedding-3-small"
)
```

```
vectorstore = Chroma(
    persist_directory="./db",
    embedding_function=embedding
)
```

```
question = "RAG란 무엇인가?"
```

```
# LLM이 다양한 질문 생성
```

```
prompt = f"""
```

```
다음 질문을 의미가 같은 다양한 질문 4개로 바꾸세요.
```

```
질문:
```

```
{question}
"""
```

```
queries = llm.invoke(prompt).content.split("\n")
```

```
docs = []
```

```
for query in queries:
    result = vectorstore.similarity_search(query, k=2)
    docs.extend(result)
```

```
# 중복 제거
```

```
unique_docs = list({doc.page_content: doc for doc in docs}.values())
```

```
print(f"검색된 문서 수: {len(unique_docs)}")
```

내부적으로 다음 작업이 자동 수행된다.

9. LangChain MultiQueryRetriever 사용 예

LangChain에서는 MultiQueryRetriever를 제공한다.

Python

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini")

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)

docs = retriever.invoke("RAG란 무엇인가?")
```

질문



LLM



질문 여러 개 생성



각각 검색



중복 제거



결과 반환

10. 언제 효과적인가?

다음과 같은 경우 검색 성능 향상이 크다.

① 사용자가 질문을 짧게 하는 경우

RAG

FAISS

② 다양한 표현이 존재하는 경우

벡터DB

↓

Vector Database

↓

임베딩 저장소

③ 동의어가 많은 경우

GPU

↓

그래픽카드

↓

CUDA Device

④ 문서 작성자가 다른 표현을 사용한 경우 사용자는

멀티모달

이라고 질문하지만
문서에는

Vision-Language Model

이라고 작성되어 있을 수도 있다.

11. 장점

- 다양한 표현을 모두 검색할 수 있다.
- 검색 Recall이 향상된다.
- 기존 Vector DB를 변경하지 않아도 된다.
- 질문 표현이 달라도 관련 문서를 찾을 가능성이 높다.

12. 단점

① 검색 시간이 증가한다.

질문을 여러 개 생성하고 여러 번 검색해야 한다.

② API 비용이 증가한다.

LLM 호출이 추가된다.

③ 중복 문서가 많이 검색될 수 있다.

중복 제거 과정이 필요하다.

④ 질문 품질에 영향을 받는다.

LLM이 부적절한 질문을 생성하면 검색 품질이 떨어질 수 있다.

13. HyDE와 비교

구분	HyDE	Multi-Query
LLM이 생성하는 것	가상의 답변 문서	여러 개의 질문
검색 횟수	1회	여러 회
검색 범위	문서 의미 확장	질문 표현 확장
속도	보통	느림
Recall 향상	높음	매우 높음
Precision 향상	높음	보통~높음

14. 실제 활용 예

사용자 질문

RAG에서 검색 성능을 높이는 방법은?

LLM이 생성한 질문

RAG 검색 최적화 방법

Retriever 성능 향상

Vector Search 개선

검색 정확도 향상 기법

RAG Retrieval 전략

각 질문으로 검색한 결과를 합치면 기존 검색보다 더 다양한 관련 문서를 찾을 수 있다.

15. 장단점 요약

장점

- 질문을 다양한 표현으로 확장하여 검색 Recall을 높인다.
- 동의어와 유사 표현을 효과적으로 처리한다.
- 관련 문서를 더 많이 검색할 수 있다.
- 기존 RAG 구조를 변경하지 않고 적용할 수 있다.

단점

- 검색 시간이 증가한다.
- LLM 호출 비용이 추가된다.
- 중복 제거 과정이 필요하다.
- 생성된 질문의 품질에 따라 검색 결과가 달라질 수 있다.

2. Self-Query Retriever

1. Self-Query Retriever란?

Self-Query Retriever는 LLM이 사용자의 질문을 분석하여 검색어(Query)와 메타데이터 필터(Filter)를 자동으로 생성한 후 검색하는 기법이다.

즉,

질문 → 검색어 + 메타데이터 조건 생성 → Vector DB 검색

이라는 과정을 수행한다.

일반적인 Vector Search는 문서 내용만 검색하지만, Self-Query Retriever는 문서의 메타데이터까지 함께 활용하여 더욱 정확한 검색을 수행한다.

2. 기존 검색 방식

기존 RAG에서는 질문을 그대로 임베딩하여 검색한다.



예)

사용자 질문

LangChain Agent 관련 문서

문서 내용만 검색한다.

3. Self-Query 방식

사용자 질문

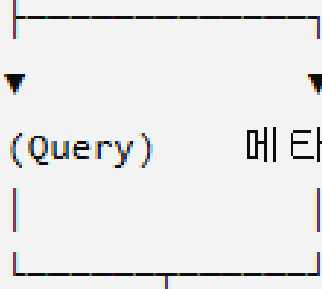


LLM



검색어 (Query)

메타데이터 (Filter)



Vector DB 검색



관련 문서

예)

사용자 질문

2024년에 작성된 RAG 논문만 보여줘

LLM이 자동 생성

검색어

RAG 논문

필터

year = 2024

4. 메타데이터란?

문서 자체가 아닌 문서의 부가 정보이다.

예)

문서	메타데이터
RAG.pdf	작성자, 날짜, 카테고리
논문	출판연도, 저널, 저자
상품	가격, 브랜드
뉴스	기자, 날짜
강의자료	Day, 난이도

예를 들어 Vector DB에는 다음과 같이 저장될 수 있다.

```
</> Python
```

```
page_content = "RAG는 ..."
```

```
metadata = {  
    "year": 2024,  
    "author": "홍길동",  
    "category": "AI"  
}
```

5. 왜 성능이 좋아질까?

기존 검색은

AI 논문

이라고 검색하면

2022년

2023년

2024년

모두 검색된다.

하지만

2024년 AI 논문

이라고 질문하면

이라고 질문하면

Self-Query는

검색어

AI 논문

+

필터

year = 2024

를 생성하여

2024년 논문만 검색한다.

6. 기존 검색 vs Self-Query

기존 검색

문서 내용만 검색

필터 없음

단순 의미 검색

속도 빠름

정확도 보통

Self-Query

문서 + 메타데이터 검색

자동 필터 생성

조건 검색 가능

약간 느림

정확도 높음

7. 동작 과정

사용자 질문



LLM



검색어 생성



메타데이터 필터 생성



Vector Search



조건에 맞는 문서 검색



최종 결과

8. 예제

사용자 질문

2025년에 작성된 LangChain 문서만 찾아줘

LLM 생성

검색어

LangChain

필터

year == 2025

Vector DB에서는

```
similarity_search(  
    query="LangChain",  
    filter={"year": 2025}  
)
```

를 수행한다.

9. 간단한 Python 예제

Python

```
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma

embedding = OpenAIEmbeddings(
    model="text-embedding-3-small"
)

vectorstore = Chroma(
    persist_directory="./db",
    embedding_function=embedding
)

docs = vectorstore.similarity_search(
    query="RAG",
    filter={"year": 2025},
    k=3
)

for doc in docs:
    print(doc.metadata)
    print(doc.page_content)
```

여기서는 filter 조건을 직접 지정했지만, Self-Query Retriever는 이 필터를 LLM이 자동으로 생성한다.

10. LangChain SelfQueryRetriever 사용 예

Python

```
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini")

retriever = SelfQueryRetriever.from_llm(
    llm=llm,
    vectorstore=vectorstore,
    document_contents="AI 문서",
    metadata_field_info=metadata_info
)

docs = retriever.invoke(
    "2025년에 작성된 LangChain 문서를 찾아줘"
)
```

내부적으로 다음 작업이 자동 수행된다.

질문



LLM



검색어 생성



필터 생성



Vector DB 검색



결과 반환

11. metadata_field_info란?

Self-Query Retriever는 메타데이터의 구조를 알아야 한다.

예)

Python

```
metadata_info = [  
    {  
        "name": "year",  
        "description": "문서 작성 연도",  
        "type": "integer"  
    },  
    {  
        "name": "category",  
        "description": "문서 카테고리",  
        "type": "string"  
    }  
]
```

LLM은 이 정보를 참고하여

```
year >= 2024
```

```
category == "AI"
```

같은 필터를 자동 생성한다.

12. 언제 효과적인가?

다음과 같은 경우 매우 효과적이다.

① 날짜 조건 검색

2025년 논문

② 작성자 검색

홍길동이 작성한 문서

③ 카테고리 검색

AI 관련 문서만

④ 가격 검색

10만원 이하 노트북

⑤ 지역 검색

서울에 있는 식당

13. 장점

- 메타데이터를 자동으로 활용한다.
- 검색 정확도(Precision)가 높다.
- 사용자가 필터를 직접 작성할 필요가 없다.
- 복잡한 조건 검색을 자연어로 수행할 수 있다.

14. 단점

① 메타데이터가 반드시 필요하다.

메타데이터가 없다면 Self-Query의 장점을 활용할 수 없다.

② LLM 호출이 추가된다.

질문을 분석하여 필터를 생성해야 한다.

③ 메타데이터 설계가 중요하다.

잘못된 메타데이터는 검색 성능을 떨어뜨린다.

④ 구현이 일반 Retriever보다 복잡하다.

15. HyDE / Multi-Query / Self-Query 비교

구분	HyDE	Multi-Query	Self-Query
LLM 역할	가상 문서 생성	여러 질문 생성	검색어와 필터 생성
검색 횟수	1회	여러 회	1회
메타데이터 활용	X	X	O
Recall 향상	높음	매우 높음	보통
Precision 향상	높음	보통	매우 높음
적합한 환경	기술문서, 논문	동의어가 많은 문서	메타데이터가 있는 문서

16. 실제 활용 예

Vector DB에 다음과 같은 메타데이터가 저장되어 있다고 가정한다.

문서	연도	카테고리	작성자
RAG 기초	2024	AI	홍길동
LangChain 심화	2025	AI	김철수
Python 기초	2025	Programming	이영희

사용자 질문

2025년에 작성된 AI 문서만 보여줘

Self-Query Retriever는 다음과 같이 해석한다.

검색어

AI 문서

필터

Python

```
{  
    "year": 2025,  
    "category": "AI"  
}
```

따라서 조건에 맞는 "LangChain 심화" 문서만 검색된다.

17. 장단점 요약

장점

- 메타데이터를 자동으로 활용하여 검색 정확도를 높인다.
- 자연어를 필터 조건으로 변환할 수 있다.
- 날짜, 작성자, 카테고리, 가격 등 다양한 조건 검색이 가능하다.
- 기업 문서 검색이나 상품 검색처럼 메타데이터가 풍부한 환경에서 매우 효과적이다.

단점

- 메타데이터가 반드시 준비되어 있어야 한다.
- LLM 호출로 인해 응답 시간이 다소 증가한다.
- 메타데이터 설계가 검색 품질에 큰 영향을 미친다.
- 일반 Similarity Search보다 구현이 복잡하다.

18. 한 줄 요약

Self-Query Retriever는 "LLM이 사용자의 자연어 질문을 분석하여 검색어와 메타데이터 필터를 자동으로 생성한 후 검색하는 기법"으로, 메타데이터를 적극 활용하여 RAG의 검색 정확도(Precision)를 높이는 대표적인 고급 검색 전략이다.



감사합니다